

TOPIC 01: API (APPLICATION PROGRAMMING INTERFACE)

API Kya Hai? (Core Concept)

API ka pura naam **Application Programming Interface** hai. Yeh do software programs ke beech ka ek aisa **secure communication bridge** hai jo unhe aapas me data aur functionalities exchange karne deta hai, bina ek dusre ke internal database ya backend logic ko jaane.

Real-Life Analogy: The Restaurant Waiter

Socho aap ek restaurant me khana khane gaye ho:

- **Customer (Frontend / Client):** Table par baithkar menu se dish ka order deta hai.
- **Kitchen (Server / Database):** Jahan actual food banta hai aur sara secret raw material store hota hai.
- **Waiter (API):** Customer se order lekar Kitchen tak pahuchata hai, aur kitchen se prepared dish laakar customer ko deliver karta hai!



Real-World API Use Cases:

- **Weather Apps:** Phone ka app **OpenWeather API** se current live temperature mangwata hai.
- **Online Payments:** Swiggy ya Amazon **Razorpay / Stripe API** ke through payment verify karte hain.
- **Google Login:** Website bina aapka Google password jaane Google Identity API se authentication kar leti hai.



Topper Insight (Abstraction): API backend ka complex logic hide karta hai. Client ko bas yeh matlab hai ki request kahan bhejni hai aur response me kya data milega!

TOPIC 02: REST (REPRESENTATIONAL STATE TRANSFER)

REST Kya Hai? (Core Definition & Origin)

REST ka pura naam **Representational State Transfer** hai. Yeh koi software, library, framework ya programming language NAHI hai!

Ise **Dr. Roy Fielding** ne saal 2000 me apne Ph.D. research paper me introduce kiya tha. Yeh ek **Architectural Style** (design principles aur rules ka set) hai jo batata hai ki internet par web services ko HTTP protocol ke standard features use karke efficiently kaise communicate karna chahiye.

Resource & Representation Ka Concept:

- **Resource:** Server par saved har entity (users, posts, photos) ek resource hai, jiska apna unique URL hota hai (e.g. /posts/101).
- **Representation:** Jab client resource mangta hai, toh server us resource ka current snapshot/state ek format me transfer karta hai (almost always JSON format).

REST Ke 6 Core Guiding Principles (Constraints):

- **1. Client-Server Architecture:** Frontend (UI) aur Backend (Database) ek dusre se completely independent hone chahiye.
- **2. Statelessness (Most Important!):** Server client ka koi session ya memory state save nahi karega. Har request me pura authentication token aur context hona chahiye.
- **3. Cacheable:** Server response me header hona chahiye jo bataye ki data client par cache ho sakta hai ya nahi (taaki server load bache).
- **4. Uniform Interface:** Consistent URLs (nouns jaise /posts) aur standard HTTP verbs (GET, POST, etc.) use hone chahiye.
- **5. Layered System:** Client ko fark nahi padta ki wo direct server se juda hai ya beech me CDN/load balancer/proxy hai.
- **6. Code on Demand (Optional):** Server chahe toh client ko execute hone wala script (e.g. JS) bhej sakta hai.

TOPIC 03: API vs REST vs RESTFUL API (THE MASTER COMPARISON)

Interview Special Comparative Table:

Feature / Parameter	API	REST	RESTful API
Core Definition	Do software ke aapas me baat karne ka interface/contract.	Web APIs design karne ke 6 architectural rules & guidelines.	Aisi working web service jo REST ke rules ko strictly follow karti hai.
Nature / Type	General concept & mechanism.	Architectural style / design philosophy.	Concrete web service implementation.
Protocols Used	Koi bhi protocol (TCP, HTTP, OS system calls, memory).	Web & HTTP architecture par based.	Strictly HTTP / HTTPS protocol use karti hai.
Data Format	Any format (JSON, XML, Plain text, Binary, Objects).	Format agnostic (focus on resource representation).	Almost universally JSON (ya XML).
Constraint Enforcement	Koi strict architectural rules nahi hote.	6 mandatory constraints define karta hai.	REST ke saare constraints code me strictly enforced hote hain.
Real-World Example	Math.random(), fs.readFile(), DOM API	Roy Fielding ke 2000 ke Ph.D. principles	GitHub REST API, Twitter/X API, Express /posts CRUD API

Golden Formula for Interviews:



"All RESTful APIs are APIs, but not all APIs are RESTful!"

(Jaise har Car ek Vehicle hoti hai, par har Vehicle Car nahi hota!)



Common Interview Mistake: Kabhi mat kehna ki REST ek protocol ya library hai! REST ek Architectural Style hai jo HTTP protocol ke upar rules define karta hai.

TOPIC 04: HTTP METHODS & CRUD OPERATIONS MAPPING

CRUD Operations & HTTP Verbs Mapping:

CRUD Operation	HTTP Verb	Example Endpoint	Description & Purpose	Safe?	Idempotent?
Create	POST	POST /posts	Server par naya resource create karta hai.	✗ No	✗ No
Read	GET	GET /posts	Data retrieve karta hai bina modify kiye.	✓ Yes	✓ Yes
Update (Full)	PUT	PUT /posts/:id	Poore resource ko naye data se replace karta hai.	✗ No	✓ Yes
Update (Partial)	PATCH	PATCH /posts/:id	Sirf passed specific fields ko modify karta hai.	✗ No	✗ No*
Delete	DELETE	DELETE /posts/:id	Resource ko permanently remove karta hai.	✗ No	✓ Yes

Sabse Bada Doubt: PUT vs PATCH me kya farq hai?

- **PUT (Complete Replacement):** Agar post me { title, content, author } hai aur aapne sirf { content: "New" } bheja, toh title aur author null/delete ho jayenge! Pura document overwrite hota hai.
- **PATCH (Partial Update):** Sirf wahi field change hogi jo bheji gayi hai (jaise sirf content), baaki title aur author bilkul safe rehte hain!

Safe vs Idempotent:



• **Safe Method:** Jo server ke data ko alter nahi karta (e.g. GET).

• **Idempotent:** Ek baar chalao ya 100 baar, server state par same effect padta hai (e.g. GET, PUT, DELETE).

TOPIC 05: STATUS CODES & TOPIC 06: RESTFUL ROUTES

Standard HTTP Status Codes in REST APIs:

- **2xx (Success):** 200 OK (Standard success), 201 Created (POST se naya record bana), 204 No Content (DELETE successful).
- **3xx (Redirection):** 301 Moved, 302 Found / Redirect (Express `res.redirect` me use hota hai).
- **4xx (Client Errors):** 400 Bad Request (Validation fail), 401 Unauthorized (Login required), 403 Forbidden (No permission), 404 Not Found (Resource missing).
- **5xx (Server Errors):** 500 Internal Server Error (Server crash / backend bug).

Standard 7 RESTful Routes for Posts Resource:

Method	Route Path	Action Name	Purpose / Kya Karta Hai	Express Action
GET	/posts	Index	Saare posts ki list display karna	<code>res.render("index.ejs")</code>
GET	/posts/new	New	Naya post create karne ka HTML form dikhana	<code>res.render("new.ejs")</code>
POST	/posts	Create	Form data accept karke naya post save karna	<code>res.redirect("/posts")</code>
GET	/posts/:id	Show	Specific ID wale single post ki details dikhana	<code>res.render("show.ejs")</code>
GET	/posts/:id/edit	Edit	Specific post ko edit karne ka form render karna	<code>res.render("edit.ejs")</code>
PATCH	/posts/:id	Update	Specific post ka content modify/update karna	<code>res.redirect("/posts")</code>
DELETE	/posts/:id	Destroy	Specific post ko DB/array se delete karna	<code>res.redirect("/posts")</code>

TOPIC 07: RESPONSE HANDLING & TOPIC 08: UUID PACKAGE

Express Response Handling: 3 Key Methods

- `res.send(data)`: String, HTML, ya JSON object bhejta hai. REST APIs me JSON data return karne ke liye use hota hai.
- `res.render("index.ejs", { posts })`: Server-side EJS template me dynamic data inject karke HTML page browser ko bhejta hai.
- `res.redirect("/posts")`: Browser ko 302 code dekar bolta hai ki naye URL par automatic GET request kare.

Post-Redirect-Get (PRG) Pattern:



POST, PATCH, ya DELETE route ke baad direct `res.render()` mat karo! Hamesha `res.redirect('/posts')` karo. Agar direct render karoge aur user ne browser Refresh (F5) kiya, toh form resubmit hokar duplicate entry ban jayegi!

Topic 08: UUID Package (Universally Unique Identifiers)

Array Index (`posts[0]`) kyu use nahi karte?

Agar user ne pehla post delete kar diya (`posts.splice(0, 1)`), toh aage ke sabhi posts ke index shift ho jayenge! Kisi ne `/posts/2` bookmark kiya hoga toh ab use dusra post dikhega!

Solution: UUID Package

- UUID ka matlab hai **Universally Unique Identifier** (128-bit random string).
- Installation: `npm install uuid`

```
uuid-usage.js

const { v4: uuidv4 } = require('uuid');

// Generate random unique ID string
const id = uuidv4();
console.log(id); // Output: "1b9d6bcd-bbfd-4b2d-9b5d-ab8dfbbd4bed"

// Used when creating new post:
const newPost = { id: uuidv4(), username, content };
```

TOPIC 09: HTML FORMS & METHOD-OVERRIDE MIDDLEWARE

HTML Ki Badi Limitation:



Standard HTML `<form>` tag sirf do hi HTTP methods support karta hai: **GET** aur **POST**! Form tag me `method="PATCH"` ya `method="DELETE"` likhne par browser use ignore karke by default **GET** request bhej deta hai!

Solution: method-override NPM Package

Yeh middleware form ki request query string me `?_method=PATCH` ya `?_method=DELETE` padhkar backend par HTTP verb ko override kar deta hai!

Installation & Express Setup:

- Terminal Command: `npm install method-override`

```
app.js (Middleware Setup)

const methodOverride = require('method-override');

// Express will look for ?_method=PATCH in query string
app.use(methodOverride('_method'));
```

EJS Form Syntax in VS Code Style:

```
views/posts/edit.ejs & index.ejs

<!-- 1. EDIT FORM (Triggers PATCH /posts/:id) -->
<form method="POST" action="/posts/<%= post.id %>?_method=PATCH">
  <textarea name="content"><%= post.content %></textarea>
  <button type="submit">Update Post</button>
</form>

<!-- 2. DELETE BUTTON (Triggers DELETE /posts/:id) -->
<form method="POST" action="/posts/<%= post.id %>?_method=DELETE">
  <button type="submit">Delete Post</button>
</form>
```

TOPIC 10: COMPLETE RESTFUL CRUD (VS CODE STYLE)

Full Express.js RESTful Server Implementation:

```
server.js - Production Ready RESTful CRUD Architecture

const express = require('express');
const app = express();
const { v4: uuidv4 } = require('uuid');
const methodOverride = require('method-override');

app.use(express.urlencoded({ extended: true }));
app.use(methodOverride('_method'));
app.set('view engine', 'ejs');

let posts = [
  { id: uuidv4(), username: "shivam", content: "Learning REST APIs!" },
  { id: uuidv4(), username: "rahul", content: "Express makes routing easy!" }
];

// 1. INDEX: GET /posts (Read all)
app.get('/posts', (req, res) => {
  res.render('posts/index.ejs', { posts });
});

// 2. NEW: GET /posts/new (Render form)
app.get('/posts/new', (req, res) => {
  res.render('posts/new.ejs');
});

// 3. CREATE: POST /posts (Add new post)
app.post('/posts', (req, res) => {
  const { username, content } = req.body;
  posts.push({ id: uuidv4(), username, content });
  res.redirect('/posts'); // PRG Pattern
});

// 4. SHOW: GET /posts/:id (Read one post)
app.get('/posts/:id', (req, res) => {
  const post = posts.find(p => p.id === req.params.id);
  res.render('posts/show.ejs', { post });
});

// 5. EDIT: GET /posts/:id/edit (Render edit form)
app.get('/posts/:id/edit', (req, res) => {
  const post = posts.find(p => p.id === req.params.id);
  res.render('posts/edit.ejs', { post });
});

// 6. UPDATE: PATCH /posts/:id (Update specific post)
app.patch('/posts/:id', (req, res) => {
  const post = posts.find(p => p.id === req.params.id);
  post.content = req.body.content;
  res.redirect('/posts');
});

// 7. DESTROY: DELETE /posts/:id (Remove post)
app.delete('/posts/:id', (req, res) => {
  posts = posts.filter(p => p.id !== req.params.id);
  res.redirect('/posts');
});

app.listen(8080, () => console.log("REST Server running on port 8080"));
```